

budget-location.ca - Étape 1 - cURL

Objectifs

- Se familiariser avec la structure du site client en PHP
- Se familiariser avec la librairie cURL de PHP
- Se familiariser avec les requêtes synchrones
- Se familiariser avec l'agrégation de données
- Se familiariser avec la base d'un "Client API" (API Wrapper)
- **Ajouter la liste de produits de l'API à celle déjà présente dans le site**

Comme analyser le code PHP (avec ou sans Xdebug)

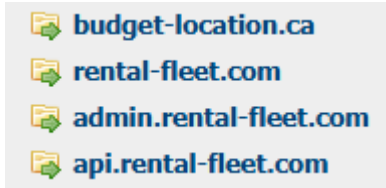
On peut voir le contenu d'une variable avec **var_dump()**, **print_r()** et **echo()** pour se débarrasser. Vous avez aussi les fonctions **pre()** et **dd()** dans **src/tools.php**.

Après le cours, configurez Xdebug si ce n'est pas déjà fait.

Voir PDF **Environnement/WAMP + Xdebug.pdf**

1- Vérifications !

- Vos 4 sites fonctionnent ? **C'est essentiel !**
- Ils utilisent les bon noms de domaine dans WAMP



- Le nom de domaine de l'API redirige bien vers localhost:3000
- L'étape #1 de l'API est complétée
- Votre API est testée par HTTPie
- Votre API retourne un JSON des 3 produits de la BD

2- Modifier la base de données de budget-location.ca

- Ajouter à la table **product** un champ **price** de type **int**, taille **4** et valeur par défaut **0**

Table product → Structure →  Ajouter 1 colonne(s) après alt 

Nom	Type	Taille/Valeurs*	Valeur par défaut
price	INT	4	Tel que défini : 0

- Faites les correctifs pour obtenir le prix dans les données des produits.
Pour le moment nous n'allons que modifier **selectAll** et **selectByTitle** de **src/ProductDAL.php**
- Modifiez l'affichage HTML pour **afficher le prix**, placez-le sous la **description**

3- Librairie RentalFleet

- Créer un dossier **RentalFleet** sous **vendor**

Ceux qui conçoivent une API, développent souvent une petite librairie pour faciliter l'intégration des appels API pour les clients.

On appelle ce petit logiciel un "Client API" ou "API Wrapper" ou "SDK API".

En PHP, le standard est d'ajouter les librairies externes sous le dossier **vendor**.

4- Base de Client API

- Ajouter dans le dossier **RentalFleet** le fichier **Curl.php**

[PHP: cURL - Manual](#)

cURL est une librairie de PHP qui permet de faire des requêtes HTTP.

Les requêtes sont synchrones, donc elles attendent la réponse avant de continuer à exécuter le code PHP.

Certains paramètres de l'appel cURL permettent de mettre fin à la requête après un certain délai.

Ce délai est nécessaire afin de compléter la tâche dans un temps raisonnable et offrir une qualité de service à l'utilisateur ou au processus.

5- Exceptions personnalisées

- Créer dans le dossier **RentalFleet** un fichier **exceptions.php** contenant 3 exceptions personnalisées

En PHP, le mot clé **extends** permet d'hériter d'une classe. [PHP: Étendre les Exceptions - Manual](#)

- Ces exceptions sont utilisées dans la fonction **Curl::send**, allez voir leurs noms

6- Ajuster Curl::send

- Ajouter l'entrée dans le tableau `$requestHeaders` pour que les valeurs `application/json` et `text/plain` soit incluse comme valeur de l'en-tête `accept`

Actuellement la fonction **send** fonctionne pour la méthode GET, sauf pour la variable **\$requestHeaders** qui est actuellement vide. Ce tableau contient est la liste des en-têtes qu'on veut joindre à la requête.

Comme nous **souhaitons** recevoir une réponse en JSON ou en texte, on doit le mentionner grâce à l'en-tête **accept**.
[En-tête Accept - HTTP | MDN](#)

Une en-tête HTTP est une **chaîne de caractères** avec la structure → **[nom en-tête]: [valeur en-tête]**

```
Accept-Language: en-US,en;q=0.5
```

Elle à comme structure le nom de l'en-tête comme telle, suivi de deux points ":", ainsi qu'une valeur.

- Le nom de l'en-tête est avant les 2 points ":"
- La valeur peut être une valeur unique.
- La valeur peut être une liste d'éléments séparés par des virgules.
- Certaines en-têtes peuvent aussi avoir une valeur construite autrement

Attention !

Ne pas confondre avec du JSON, une en-tête est une chaîne de caractères → **"Cache-Control: max-age=0"**

5- Utilisation Curl

Dans **template/product.php** on récupère les produits de la base de données du site.

Nous allons combiner les produits de **budget-location.ca** avec les produits provenant de **api.rental-fleet.com**.

Pour se simplifier la vie et pour récupérer les éléments de la réponse provenant de **Curl::send**, on utilise le concept de **décomposition de tableaux (array destructuring)** en variables.

À la fin, on possède 3 variables qui nous permettent de traiter la réponse.

```
$response = Curl::send($url);
```

```
[$status, $headers, $body] = array_values($response);
```

\$status → code de statut HTTP

\$headers → tableau associatif d'en-têtes

\$body → tableau associatif de données

À faire pour utiliser Curl::send et récupérer les données de l'API

- À la suite du code qui récupère les produits de la base de données de budget-location.ca

```
if (!empty($search)) {  
    $search = "%" . strtolower($search) . "%";  
    $products = ProductDAL::selectByTitle($connexion, $search);  
}  
else {  
    $products = ProductDAL::selectAll($connexion);  
}
```



- Vous devez utiliser **Curl::send** pour faire l'appel à l'API
- Vous devez attrapper (try/catch) tous les types d'exceptions mais nous ne ferons rien d'autre que relancer l'exception pour le moment :

```
catch (Exception $e) {  
    throw $e;  
}
```

- Vous devez valider que le code de statut HTTP est 200
- Vous devez valider l'entête de retour **content-type** est du JSON

Note : Curl::send converti tous les textes d'en-tête de retour en minuscule

Pour chaque produit provenant de l'API

- vous devez **ajouter les produits à la liste existante \$products**

Cet ajout s'appelle une **Agrégation de données**, on fusionne les 2 listes.

Champs de la BD locale : id, title, description, image, alt, price

Champs de l'API : id, name, picture, price

Certaines données n'auront pas d'association directe, certaines seront une chaîne vide.

Nous devons décider de comment fusionner les données selon le contexte.

- La vue actuelle ne doit pas être affectée par les changements
- On voudrait aussi savoir quelle donnée vient de l'API
donc ajoutez une entrée 'source' => 'api' pour l'indiquer à chaque produit ajouté à **\$products**

```
$products[] = [  
    'id' => ?,  
    'title' => ?,  
    'description' => '',  
    'image' => ?,  
    'alt' => '',  
    'price' => ?,  
    'source' => 'api'  
];
```

À ce point, **on veut au minimum voir le nom des 3 produits et le prix à la suite de ceux déjà présents dans le site**, nous pourrions ajuster plus tard l'agrégation car on voit que les champs ne correspondent pas tous.

 Chariot Vivamus leo tortor, finibus ut porta ac, placerat sed arcus. 0 \$	 Couverture Esse in voluptate exercitation ea. Lorem sit duis ipsum voluptate ullamco eu cillum eu occaecat dolore. 0 \$	Hand Cart 118 \$
Moving Blanket 24 \$	Packaging Tape 19 \$	

6- Création de constantes

Pour rendre notre code plus robuste, nous allons créer des constantes qui représentent les textes qui seront souvent utilisés à travers le développement de l'intégration avec l'API. [PHP: Les constantes - Manual](#)

Créer dans **RentalFleet** un fichier **constants.php** qui contiendra :

- la liste des en-têtes HTTP utilisées jusqu'à maintenant : HTTP_HEADER_?
- la liste des types de contenu utilisés jusqu'à maintenant : CONTENT_TYPE_?
- la route utilisée jusqu'à maintenant : ROUTE_?
- le domaine (url) de l'API : API_DOMAIN

Complétez les noms des constantes, le plus proche de la valeur réelle afin que ce soit bien clair.

7- Utilisation des constantes

- Modifiez **Curl::send \$requestHeaders** pour utiliser les constantes dans la définition de l'en-tête
- Modifier **template/product.php** pour utiliser les constantes ou c'est nécessaire

BONUS

- Créer un fichier **tools.php** sous **RentalFleet**
- Créer une fonction utilitaire qui va créer l'entrée d'en-tête
 - accepte un texte d'en-tête
 - accepte une liste de valeur
 - retourne la chaîne représentant le texte complet de l'en-tête
- Utilisez votre fonction dans **Curl::send \$requestHeaders**